Flight Software Challenge Brief

On subsequent pages you will find a data sheet for a fictional subsystem, the Optical Operations Pointing System (OOPS). It details the mode of operation and control protocol for interfacing with it.

You should also receive a file containing some sample data. This file should be interpreted as serial data that has been “sniffed” from a shared communications bus between a control computer and a number of OOPS systems. The file is formatted as a series of whitespace-separated hexadecimal numbers representing serial bytes in the communications stream.

The sample data is designed to emulate the conditions of a realistic system, meaning there may be mistakes or corruptions, or nonsense data between packets in the data that should be accounted for in your code. Please state any assumptions that you have made about either the sample data or the datasheet.

Using the datasheet as a reference, please prepare a software project that can parse the sample data and answer the following questions:

1. How many valid packets are present in the sample data?
2. Of the invalid packets, why are they invalid? Please detail reasons and numbers for each reason
3. How much of the data in the stream is not useful data, e.g. padding between packets?
4. Construct a telecommand packet to OOPS ID 4 with reserved bytes 21 A2 and data content of 07 DF EA 23 49 04 EC 22 62. What is its checksum?

The project can be written in a language of your choice, but we would encourage you to use a language that you think is relevant to the job role.

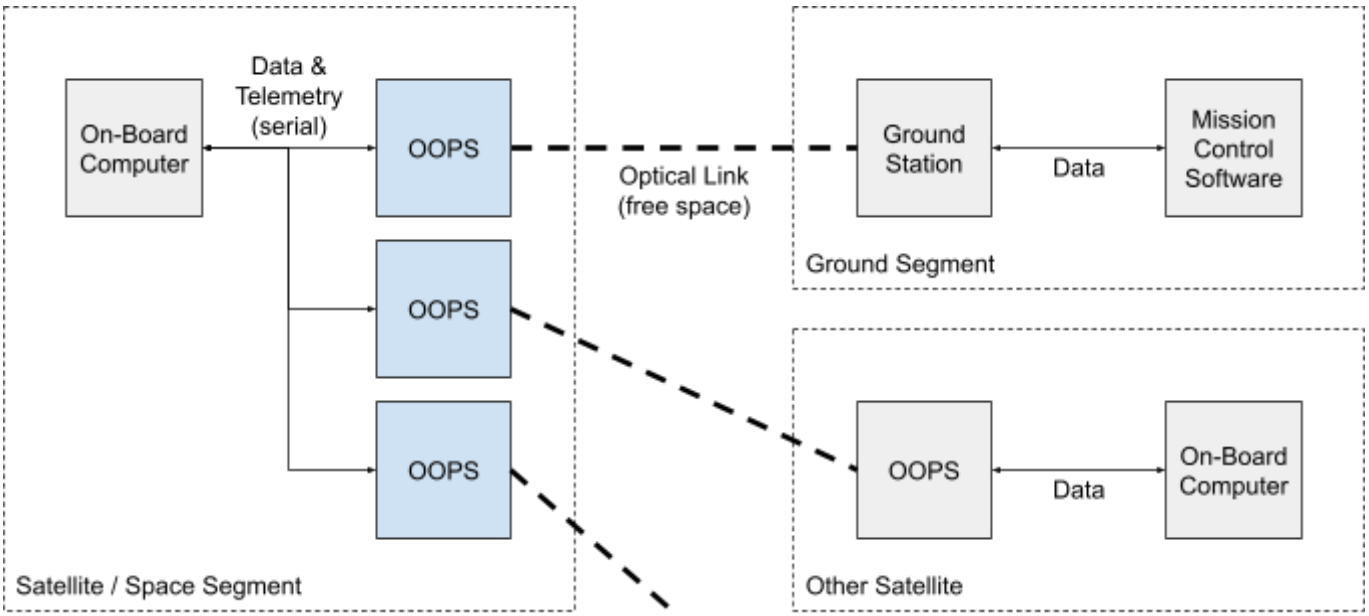
Please bear in mind that during the challenge interview you will be asked further questions which will likely require you to make modifications to your code.

Optical Operations Pointing System (OOPS) Datasheet

The OOPS is an optical free-space communications subsystem for use on satellite platforms. It allows the sending and receiving of data through an optical connection to another OOPS system or a ground station.

The OOPS allows a controlling subsystem to send and receive data over the link, as well as receive telemetry information about the state of the subsystem and the link.

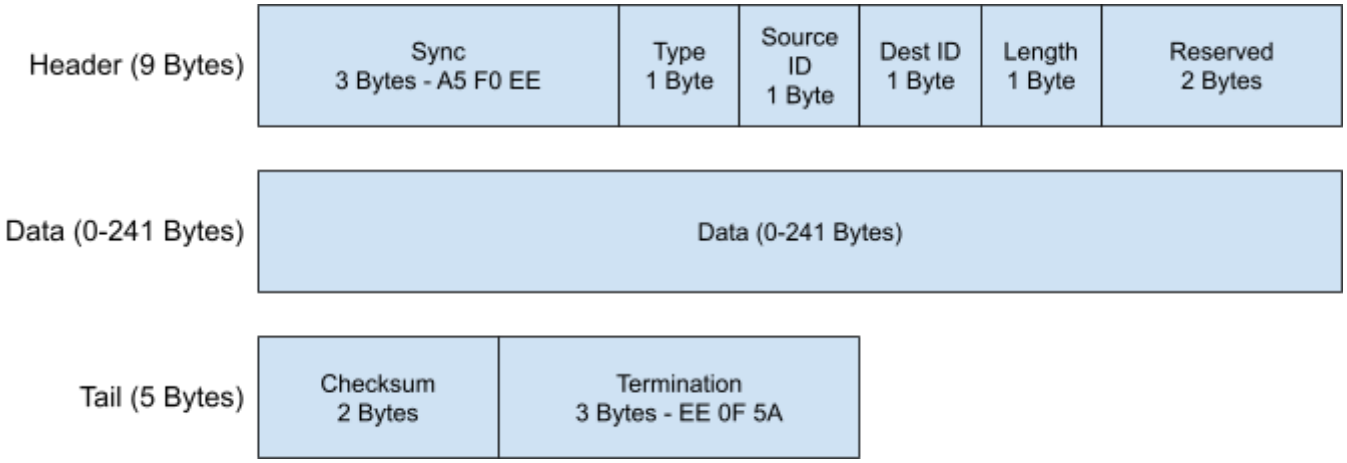
Mode of Operation



The on-board computer acts as the controller for the bus. It is responsible for issuing commands to the subsystems and parsing data reported by them.

Everything connects to a multi-drop bus, meaning all systems hear all packets and are responsible for differentiating whether each given packet is “theirs”.

Packet format (13-255 bytes)



- Header (9 bytes)
  - Sync Marker (3 bytes) A5 F0 EE
    - Used to track the start of a packet
  - Packet Type (1 byte)
    - See “packet types” section below
  - Source Device ID (1 byte)
    - Device ID of the sending device
  - Destination Device ID (1 byte)
    - Device ID of the intended recipient (or the special “broadcast” ID, see “source and destination ID” section below)
  - Reserved (2 bytes)
    - Reserved for future use
  - Data Length (1 byte)
    - Length of the data field, in bytes
- Data (0-241 bytes)
  - Data content of the packet, exact content meaning will vary based on the “packet type” field
- Packet Tail (5 bytes)
  - Checksum (2 bytes)
    - See “Checksum” section below
  - Termination bytes (EE 0F 5A)

Packet Types

Type 00 - Null Packet
Packets with the NULL type (00) should be ignored. No guarantees are made about the correctness of the packet or its contents, or the validity of the checksum field

Type 01 - Request Packet
Request packets are issued by the on-board computer to request or command the OOPS system into different modes or configurations

Type 02 - Telemetry Packet
Telemetry packets are sent by OOPS systems upon request, containing data requested from the system.

Type 03 - Inbound Data Packet
Inbound data packets contain data received by OOPS systems on the optical interface, which the OOPS has decoded and is forwarding to the control computer.

Type 04 - Outbound Data Packet
Outbound data packets are sent by the control computer to a specific OOPS system, to be sent over the optical link.

Type 05 - Command Packet
Command packets are sent by the control computer to OOPS systems to command them to perform certain actions or change modes

Type 06 - Ack/Nack Packet
Ack/Nack packets are sent as responses to commands or requests for telemetry and report the success or failure of each packet.

Source and Destination ID

The controller (your system) should always use ID 1. It must always send packets using source ID 1, and should receive and parse all packets with destination ID 1.

OOPS systems will use Source Device IDs 4, 5, 6 and 7 in the same manner.

Messages to the special destination ID 255 are broadcast packets. All systems should receive and handle these messages.

Checksum

The packet structure contains 2 checksum bytes at the tail of the packet, which are used to validate the contents of the packet. The checksum should be applied to the entirety of the packet, **excluding** the head sync marker, checksum bytes and tail sync marker.

The checksum algorithm is as follows:

- Start with an 16-bit accumulator, initialised to the starting value 0xACE1
- For each byte in the message:
    - Rotate the accumulator left by 3 bits.
    - XOR the low byte of the accumulator with the new byte.
    - Add the value **0x1F** to produce the iterated checksum (wrapping at the 16-bit unsigned limits)
- After applying this process, the contents of the accumulator is your checksum.